

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Практическая работа

Выполнил:

Андронов Денис

Группа

К3342

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Д34: Технический дизайн микросервисной архитектуры

Срок: 01.04.2026

Задание:

1. Разделите ваше монолитное бэкэнд-приложение на микросервисы, придерживаясь концепции database-per-service
2. Спроектируйте микросервисную архитектуру: отразите взаимосвязь между микросервисами, опишите способы их взаимодействия друг с другом
3. Спроектируйте разделение вашей БД на обособленные БД
4. Спроектируйте и опишите эндпоинты, необходимые для обеспечения межсервисного взаимодействия в формате спецификации OpenAPI
5. Опишите варианты запросов и ответов, задокументируйте возможные ошибки
6. В результате вы должны сформировать документ, в котором будут описаны конкретные требования и шаги для достижения разделения вашего монолитного бэкэнд-приложения на отдельные микросервисы

Ход работы

Монолитное приложение системы бронирования ресторанов разделяется на три независимых микросервиса по предметным областям (Domain-Driven Design).

1. Auth Service (Сервис аутентификации)

- Зона ответственности: Регистрация, авторизация, управление учетными записями, выдача и валидация JWT-токенов.

2. Catalog Service (Сервис каталога ресторанов)

- Зона ответственности: Управление информацией о ресторанах, регионах, городах и отзывах. Поиск и фильтрация заведений.

3. Booking Service (Сервис бронирования)

- Зона ответственности: Создание бронирований, проверка доступности времени, управление статусами броней.

Согласно паттерну Database-per-Service, каждый микросервис получает свою физически или логически изолированную базу данных PostgreSQL. Сервисы не могут делать прямые SQL-запросы (JOIN) к чужим таблицам - обмен данными происходит только через API.

БД auth_db:

- Таблица users (id, email, password_hash, role, first_name, last_name).

БД catalog_db:

- Таблица regions (id, name).
- Таблица cities (id, name, region_id).
- Таблица restaurants (id, name, city_id, address, price_category, cuisine).
- Таблица reviews (id, restaurant_id, user_id, text, rating). (user_id здесь - это просто число, без внешнего ключа (Foreign Key) к БД auth_db).

БД booking_db:

- Таблица reservations (id, restaurant_id, user_id, reservation_time, guests_count, status). (Аналогично: restaurant_id и user_id хранятся как простые числа).

Отказ от жестких внешних ключей на уровне БД компенсируется валидацией данных на уровне логики микросервисов при межсервисном взаимодействии.

Взаимодействие микросервисов

Выбран **синхронный способ взаимодействия** по протоколу HTTP/REST. Для защиты внутренних эндпоинтов от внешних

пользователей предполагается использование закрытого внутреннего контура сети (или проверка специального внутреннего токена сервиса).

Сценарий взаимодействия при создании бронирования:

1. Клиент отправляет POST-запрос на создание брони в Booking Service (приложив JWT-токен клиента).
2. Booking Service отправляет внутренний запрос в Auth Service, чтобы подтвердить валидность токена и получить ID пользователя.
3. Booking Service отправляет внутренний запрос в Catalog Service для проверки: существует ли такой restaurant_id и работает ли ресторан в запрошенное время.
4. Если оба сервиса отвечают 200 OK, Booking Service сохраняет запись в booking_db.

При межсервисных запросах могут возникать ошибки. В коде они будут обрабатываться через блоки try-catch в связке с HTTP-клиентом (например, Axios).

Код	Описание ошибки	Стратегия обработки
400	Bad Request (Неверный формат запроса)	Возврат ошибки клиенту с указанием неверных полей.
401	Unauthorized (Невалидный JWT токен)	Отказ в операции (Booking Service сразу возвращает 401 клиенту).
404	Not Found (Ресторан или Юзер удален)	Отмена транзакции. Возврат клиенту

		читаемой ошибки "Ресторан больше не доступен".
500/503	Service Unavailable (Один из сервисов упал)	Сообщает клиенту: "Сервис временно недоступен, повторите попытку позже", предотвращая каскадное падение всей системы.

Требования и шаги для реализации

1. **Настройка инфраструктуры:** Создать три отдельные папки/репозитория для каждого сервиса с собственными package.json и tsconfig.json.
2. **Изоляция БД:** Настроить три разных AppDataSource в TypeORM для каждого сервиса, создав три базы в pgAdmin.
3. **Рефакторинг Entities:** Удалить декораторы @ManyToOne и @OneToMany, связывающие разные предметные области (например, убрать связь User из Reservation). Заменить их на обычные колонки @Column() userId: number.
4. **Разработка внутреннего API:** Написать контроллеры для маршрутов /internal/....
5. **Интеграция HTTP-клиента:** Добавить библиотеку axios для выполнения запросов между сервисами.

Вывод

В результате данной работы я спроектировал разделение монолитного приложения на сервисы.